Google Cloud Data-Source Governance — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary — Google Cloud Data-Source Governance: Microsoft Purview 4

What you get . . . 4

What this delivers . . . 4

The deployment model . . . 4

Outcome . . . 5

1. Solution Overview — Google Cloud Data-Source Governance: Microsoft Purview 5

What this solution is . . . 5

Why this matters . . . 5

The Google Cloud source this covers . . . 5

The Data Map, collections, and roles . . . 6

Credentials and network — safe by design . . . 6

Licensing and the evolving portal . . . 6

What this is not . . . 6

The deployment model . . . 7

2. Requirements Guide — Google Cloud Data-Source Governance: Microsoft Purview 7

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription . . . 7

2.2 Google BigQuery connector availability confirmed for your tenant . . . 7

2.3 Least-privilege data-governance roles on the Data Map collection . . . 8

2.4 A Google Cloud inventory of the BigQuery projects/datasets in scope, with owners . . . 8

2.5 A Google Cloud service account with read-only BigQuery access (key stored in Key Vault) . . . 8

2.6 A self-hosted integration runtime (SHIR) with the BigQuery driver prerequisites . . . 8

2.7 A classification approach (system + any custom classifications) . . . 9

2.8 A pilot scope and change control . . . 9

Checklist . . . 9

Global Requirements — Microsoft Purview Solutions 10

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 10

2. Purview roles (least-privilege) and portal access . . . 10

3. Security & Compliance PowerShell + Graph toolchain . . . 11

4. Data sources & scope in scope . . . 11

5. Compliance, records, privacy & legal stakeholders . . . 11

6. A pilot scope and change control . . . 11

3. Implementation Guide — Google Cloud Data-Source Governance: Microsoft Purview 12

Step 1 — Assess current state (read-only) . . . 12

Step 2 — Design the collection and role model . . . 12

Step 3 — Establish the BigQuery service-account credential (safe by design) . . . 13

Step 4 — Establish the SHIR and network path . . . 13
Step 5 — Record the pilot scope . . . 13
Step 6 — Register the pilot Google Cloud source into the Data Map . . . 13
Step 7 — Configure and pilot the scan (bounded scope, pilot-first) . . . 14
Step 8 — Review scanned assets and classifications . . . 14
Step 9 — Assign ownership on the Google Cloud assets . . . 14
Step 10 — Validate against the pilot gate . . . 14
Step 11 — Expand across the Google Cloud estate (enforce) . . . 14
5. Testing Guide — Google Cloud Data-Source Governance: Microsoft Purview 15
Test 1 — Account and roles are ready . . . 15
Test 2 — The Google BigQuery connector is available . . . 15
Test 3 — The service-account credential path works (Key Vault + Test connection) . . . 15
Test 4 — The SHIR and network path work . . . 15
Test 5 — The scan succeeds and returns assets . . . 16
Test 6 — Google Cloud assets are classified . . . 16
Test 7 — Scanned Google Cloud assets have owners . . . 16
Test 8 — Consolidated readiness gate . . . 16
Pass/fail gate . . . 16
6. Troubleshooting Guide — Google Cloud Data-Source Governance: Microsoft Purview 16
A. Cannot reach the account / token fails . . . 17
B. Google BigQuery source registration or Test connection fails . . . 17
C. Scan runs but classifies nothing . . . 18
D. Classifications look wrong . . . 18
E. Ownership / catalog issues . . . 18
F. SHIR health . . . 19
G. Cost surprises . . . 19
H. Diagnostics quick reference . . . 19
I. When to escalate . . . 19
7. Best Practices — Google Cloud Data-Source Governance: Microsoft Purview 19
8. Deliverables Summary — Google Cloud Data-Source Governance: Microsoft Purview 20
Documentation . . . 20
Named artifacts . . . 21
PowerShell scripts . . . 21
What the buyer receives . . . 22
Outcome . . . 22
Appendix A — PowerShell Scripts 23

Executive Summary — Google Cloud Data-Source Governance: Microsoft Purview

Most data-governance efforts stop at Microsoft 365. But some of the data that carries the most risk — customer records, financial extracts, analytics tables — lives in **Google Cloud**, outside M365, ungoverned and unclassified. You cannot protect, retain, or grant access to data you cannot see. This solution implements a focused slice of the **data-estate** side of Microsoft Purview: **registering, scanning, and classifying Google BigQuery in the Purview Data Map** — so Google Cloud data becomes a mapped, classified, owned part of your governed estate, not a blind spot.

This is a complete, do-it-today package written for an engineer standing up the Google Cloud connector in Purview Data Governance for the first time. Every artifact is named once and reused everywhere, the service-account key lives in Azure Key Vault (never in code), and every scan is proven on a bounded pilot before the estate is touched. Because the Purview Data Governance experience is evolving and its features are consumption-billed, the guide cites the method and flags every capability "verify your licensing/tenant" rather than asserting a path, an IAM role, or a price as permanent.

What you get

• A plain-language Solution Overview of the Google BigQuery connector workflow — the Data Map, collections and roles, the service-account credential path, the self-hosted integration runtime, and scanning + classification — and how it complements Microsoft 365 Purview.

• A Requirements Guide with verify / implement-if-missing for every prerequisite: the Purview account, connector availability, least-privilege collection roles, BigQuery projects/datasets with owners, a read-only service account with its key in Key Vault, the SHIR and network path, a classification approach, and the pilot scope.

• An Implementation Guide of precise, ordered steps: design the collection/role model, establish the service-account credential and SHIR safely, register and **pilot-scan** a bounded BigQuery scope, review classifications, assign ownership, validate against the gate, then expand with scheduled scans.

• Five PowerShell tools for readiness/role discovery, pilot-scope capture, source registration and scan configuration (`-WhatIf` by default), a scan-and-classification coverage report, and a consolidated readiness gate — authentic Purview data-plane REST automation with **no credentials in code**.

• Testing and Troubleshooting guides with explicit pass/fail gates, plus field-proven Best Practices.

What this delivers

Google Cloud data mapped into the Purview Data Map: BigQuery datasets and tables scanned and classified, with named owners and a pilot-proven scan pattern — validated on a bounded scope before you scale.

The deployment model

Assess → Design → Configure → **Pilot (bounded scan)** → Validate → Expand — with a pilot gate before any estate-wide scanning.

Outcome

Google Cloud data discoverable, classified, and owned in the same governed map as the rest of the estate — far beyond Microsoft 365 — with regulatory, records, and access determinations left to the owners who make them.

1. Solution Overview — Google Cloud Data-Source Governance: Microsoft Purview

What this solution is

Microsoft Purview has two related but distinct sides. The **compliance** side (labels, DLP, retention, insider risk) governs Microsoft 365 content. This solution implements a focused slice of the **data governance** side: **registering, scanning, and classifying your Google Cloud data into the Purview Data Map**, so Google Cloud data stops being a governance blind spot and becomes a mapped, classified, cataloged part of your estate. It does not re-implement the whole Data Map / Unified Catalog build; it delivers the **Google Cloud source connector workflow** end-to-end: register the source, prove the service-account trust/credential path, run a bounded pilot scan, review classifications, then expand.

> ***Boundary.*** *This solution provides the technical method to *register, scan, classify, and catalog* Google Cloud sources. It does **not** make the business or regulatory determinations — which data is "confidential", what the retention or records schedule is, or who may access it — those belong to the organization's data owners, stewards, compliance, privacy, and legal functions. The solution gives them governed, discoverable Google Cloud data to make those decisions against.*

Why this matters

Most organizations govern Microsoft 365 well and their Google Cloud estate not at all. Analytics land in a BigQuery dataset, an application replicates a customer table into a warehouse, an analyst materializes an extract — and none of it appears in the data map. Nobody can answer "is there PII in our BigQuery datasets?", "which tables hold regulated data?", or "who owns this dataset?" — because Google Cloud data was never registered, scanned, or classified. Ungoverned data is un-securable data: you cannot protect, retain, or grant access to something you cannot see. This solution brings Google Cloud into the same Purview Data Map that already governs the rest of the estate.

The Google Cloud source this covers

The documented Google Cloud data source in the Microsoft Purview Data Map is **Google BigQuery** — Google Cloud's managed data warehouse. This solution stands up the **Google BigQuery** connector workflow end-to-end. Availability of specific multicloud connectors and their exact configuration is **evolving** in Microsoft Purview — every capability below is flagged "verify your licensing/tenant", and any additional Google Cloud source types you need (beyond BigQuery) must be **confirmed as registerable in your tenant** before you plan for them rather than assumed.

Google BigQuery — projects, datasets, tables, and views. Purview connects to BigQuery using a **Google Cloud service account**: you create a service account in the project, grant it **read-only** BigQuery access, and generate a **service-account key**. The key's **private key** is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name** — no key material is ever placed in a script or document. Scanning enumerates projects, datasets, tables, and views and applies **classifications** (system classifications such as credit-card or national-ID number, plus any custom classification you define).

The Data Map, collections, and roles

Google Cloud sources register into the **Data Map** under a **collection** hierarchy that also carries access control. Each collection carries its own role assignments: **Collection admins** (manage the collection and its roles), **Data source admins** (register sources and run scans), **Data curators** (edit catalog metadata and classifications), and **Data readers** (browse the catalog). Least-privilege role assignment is a first-class step, not an afterthought — the Implementation Guide names each role at each stage.

Credentials and network — safe by design

Scanning reaches into live Google Cloud systems, so it needs a trust path and network line of sight — never a secret in a script. This solution follows the safe-by-design pattern Microsoft documents:

• **Google BigQuery** authenticates with a **Google Cloud service account**. You generate the service-account key in Google Cloud, then store its **private key** in an **Azure Key Vault** linked to the Purview account and register it as a Purview credential **by name** (for example `DGGCP-Cred-BQ-KV`). **No key material is stored in a script or document.**

• BigQuery scans run through a **self-hosted integration runtime (SHIR)** — a machine you control with network line of sight to Google Cloud and the **BigQuery JDBC driver prerequisites** installed (historically a **Java Runtime Environment / JDK**; **verify the current driver and Java prerequisites** for your tenant). The SHIR is where the scan executes.

• Scans are always **piloted on a bounded scope first** (one project / one or two datasets) before any estate-wide scan.

Licensing and the evolving portal

Microsoft Purview Data Governance (the Data Map and its multicloud connectors) is billed through a **consumption / pay-as-you-go** model tied to an **Azure subscription**, and the capability set is evolving — the **Google BigQuery** connector, the exact service-account/credential configuration, and the SHIR/driver prerequisites may be in preview or may have changed name, path, or availability. **Verify current pricing, availability, required GCP IAM roles, and portal paths for your tenant** in Microsoft's documentation and the Purview portal before you plan. This solution cites the method; it never asserts a price, a GCP project ID, or a portal path as permanent.

What this is not

This is the *Google Cloud data-source governance* solution. It is **not** the full Data Map / Unified Catalog build, and it is **not** the Microsoft 365 compliance configuration (sensitivity/retention labels, DLP, insider risk) — those are separate Purview solutions in this category. It complements them by pulling Google

BigQuery into the same governed map.

The deployment model

> *Assess Google Cloud readiness (project, service account, connector availability, SHIR/network), design the collection and role model and the service-account/credential path, configure the Google Cloud source registration, **pilot scanning on a bounded scope** (one project / a couple of datasets), validate the results against the gate, then expand across the Google Cloud estate — moving scans from one-time to scheduled only after the pilot passes. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.***

2. Requirements Guide — Google Cloud Data-Source Governance: Microsoft Purview

Verify every requirement before you register a Google Cloud source or run a scan. Each entry gives Description, Why, How to verify, and How to implement if missing. Category-wide prerequisites (tenant, roles, toolchain, scope, stakeholders, pilot) are in `global-requirements-purview.md` — this guide adds the prerequisites specific to **governing Google BigQuery in the Purview Data Map**.

> ***Licensing/availability caveat.** Microsoft Purview Data Governance is billed by a consumption / pay-as-you-go model on an Azure subscription and is evolving; the **Google BigQuery** multicloud connector, the required GCP IAM roles, and the exact service-account/credential and SHIR/driver configuration may be in preview or may have changed. Where this guide names a capability, a GCP project ID, an IAM role, or a portal path, **verify its current availability for your tenant** in Microsoft's documentation and the Purview portal before relying on it.*

2.1 A Microsoft Purview account (Data Map) provisioned on an Azure subscription

Description: A Microsoft Purview account — the Data Map resource — provisioned in an **Azure subscription** (distinct from the M365 compliance portal). The account exposes the collection hierarchy, scanning, and the multicloud connectors Google Cloud sources register into. Why: The Data Map and scans live in this account; without it there is nothing to register Google Cloud sources into, and the multicloud connectors are surfaced by this account. How to verify: The account is visible in the Azure portal and its data-plane endpoint (`https://<account>.purview.azure.com`) resolves; run `DGGCP-Discovery.ps1` to confirm reachability. How to implement if missing: Provision a Microsoft Purview account under change control on the target subscription; confirm the consumption billing model and region with your Azure owner.

2.2 Google BigQuery connector availability confirmed for your tenant

Description: Confirmation that the **Google BigQuery** source connector is available in your Purview account (and confirmation of any additional Google Cloud source types you intend to use — do not assume a connector exists). Why: Multicloud connector availability is evolving/preview; you cannot register a source type the connector doesn't currently offer in your tenant. How to verify: In the Purview portal, **Data Map** → **Sources** → **Register**, confirm **Google BigQuery** appears as a registerable source type; check Microsoft's current multicloud-connector documentation. How to implement if missing: If the connector isn't

offered, verify licensing/region and Microsoft's current availability; scope the pilot to the source type actually available and defer the rest.

2.3 Least-privilege data-governance roles on the Data Map collection

Description: Role assignments at the right scope — **Collection admin**, **Data source admin**, **Data curator**, **Data reader** — on the collection the Google Cloud sources will register into. Azure **Owner/Contributor** on the account only where account-level changes are required. Why: Purview uses fine-grained, collection-scoped roles; registering a source and running a scan requires **Data source admin** on that collection, and least-privilege is a governance requirement in its own right. How to verify: In the Purview portal, open the collection's **Role assignments**; run `DGGCP-Discovery.ps1` (reports collections and role coverage, read-only). How to implement if missing: Assign the least-privilege role at the narrowest collection scope under change control; reserve account-level Collection admin for a small named group.

2.4 A Google Cloud inventory of the BigQuery projects/datasets in scope, with owners

Description: A written list of the **Google Cloud resources** to govern — the **BigQuery project(s)**, the **datasets** and **tables/views** within them (with region/location) — each with a named **data owner** and **data steward**, and the GCP **project ID(s)** they live in. Why: You register, scan, and classify per source; classification and ownership are meaningless without a known owner per dataset, and the project/location drive the IAM and network the scan needs. How to verify: For each in-scope project/dataset you can name the resource, its location, and its owner; run `DGGCP-Set-Scope.ps1` to record the pilot source set. How to implement if missing: Inventory the Google Cloud estate with the GCP project owners, and assign a data owner and steward to every project/dataset before registering it.

2.5 A Google Cloud service account with read-only BigQuery access (key stored in Key Vault)

Description: A **Google Cloud service account** in the in-scope project, granted **read-only** BigQuery access — historically the **BigQuery Data Viewer** and **BigQuery Metadata Viewer** roles plus the ability to run read jobs (**BigQuery Job User** / read sessions); **verify the current required IAM roles** for the Purview BigQuery connector. You generate a **service-account key** for it, and its **private key** is stored in an **Azure Key Vault** linked to the Purview account and referenced **by name** (for example `DGGCP-Cred-BQ-KV`). **No key material is ever placed in a script or document.** Why: Purview authenticates to BigQuery as this service account; read-only IAM is the least privilege a scan needs, and storing the private key in Key Vault (referenced by name) is how the provided scripts stay secret-free. How to verify: The service account exists with the read-only BigQuery roles and the **BigQuery API enabled** in the project; a Key Vault is linked to the Purview account and holds the private key; the source's **Test connection** in Purview succeeds using the named credential. How to implement if missing: Create the service account, grant the read-only BigQuery roles, enable the BigQuery API, generate the key, store its private key in Key Vault, register it as a Purview credential by name, and run **Test connection**.

2.6 A self-hosted integration runtime (SHIR) with the BigQuery driver prerequisites

Description: A **self-hosted integration runtime (SHIR)** — a machine you control, registered to the Purview account — with **network line of sight to Google Cloud / BigQuery** and the **BigQuery JDBC driver prerequisites** installed (historically a **Java Runtime Environment / JDK**; **verify the current driver and Java version** for your tenant). BigQuery scans execute on this SHIR. Why: The BigQuery connector runs its scan through a SHIR (not the Azure managed runtime); without a Running SHIR that can reach BigQuery and has the driver prerequisites, **Test connection** and scans will fail. How to verify: The SHIR shows **Running/Available** in the portal, Java/driver prerequisites are installed on the SHIR host, and **Test connection** on the BigQuery source succeeds; `DGGCP-Discovery.ps1` reports registered runtimes. How to implement if missing: Install and register a SHIR with line of sight to BigQuery, install the required Java/driver prerequisites on the host, confirm it shows Running, then re-test.

2.7 A classification approach (system + any custom classifications)

Description: A decision on how Google Cloud assets will be classified: rely on the **built-in system classifications** (credit card, national ID, and the large built-in set) and define any **custom classification rules** the organization needs (for example an internal customer-ID pattern) via the scan rule set. Why: Classification is what turns the raw BigQuery inventory into a sensitivity-aware map that drives ownership, access, and downstream protection. How to verify: System classifications appear on a test scan result; any custom classification rule is defined and matches sample data. How to implement if missing: Start with system classifications for the pilot; define custom rules only where a real internal pattern isn't covered, and validate each against sample data before estate-wide use.

2.8 A pilot scope and change control

Description: A **bounded pilot** — one collection and one Google Cloud source (one BigQuery project / one or two datasets) — to prove registration, credential, scanning, and classification before Google-Cloud-wide rollout, plus a change-control window (scans consume resources and touch live Google Cloud systems and can incur BigQuery query/egress cost). Why: Every solution in this category follows Assess → Design → Configure → **Pilot** → Validate → Expand; scanning is resource-, cost-, and access-sensitive and is always proven on a bounded scope first. How to verify: A named pilot collection, its Google Cloud source, and a change window exist; run `DGGCP-Set-Scope.ps1` to materialize the pilot scope. How to implement if missing: Choose one well-understood, high-value dataset set as the pilot; agree the scan window and expected BigQuery cost with the source owners.

Checklist

- Purview account (Data Map) provisioned on an Azure subscription — `DGGCP-Discovery.ps1`
- Google BigQuery connector availability confirmed for the tenant
- Least-privilege collection roles assigned (Data source admin to register/scan) — `DGGCP-Discovery.ps1`
- BigQuery projects/datasets inventoried with owners, location — `DGGCP-Set-Scope.ps1`
- Service account with read-only BigQuery roles + BigQuery API enabled; private key in Key Vault, referenced by name

• SHIR Running with Java/driver prerequisites and network line of sight to BigQuery — Test connection passes

• Classification approach agreed (system + any custom)

• Pilot scope and scan change window (with expected BigQuery cost) agreed — `DGGCP-Set-Scope.ps1`

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires (E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names (e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

- **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.
- **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

• **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

• **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

• **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide — Google Cloud Data-Source Governance: Microsoft Purview

Follow these steps in order. This guide assesses Google Cloud readiness, designs the collection and role model, establishes the BigQuery service-account credential and SHIR/network path safely, registers and **pilot-scans** a bounded Google Cloud source set, reviews classifications, then expands across the Google Cloud estate. Artifact names are fixed — create them with the exact names shown and reuse them everywhere. **Every scan is piloted on a bounded scope before the Google Cloud estate is scanned.**

> *Model: prove the Google Cloud connector, credential path, and classification on a pilot scope, then expand. The repeatable way: **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand.** Verify current portal paths, connector availability, required GCP IAM roles, and the SHIR/driver prerequisites for your tenant as you go — the multicloud connectors are evolving.*

Step 1 — Assess current state (read-only)

1. Run `DGGCP-Discovery.ps1` to capture readiness signals: the Purview account endpoint is reachable,

which **collections** exist and their **role coverage**, which **integration runtimes** are registered (you will need a SHIR for BigQuery), and how many sources are already registered. No changes are made.

1. In the Purview portal (**Data Map** → **Sources** → **Register**) confirm **Google BigQuery** is offered

for your tenant.

1. Record gaps against the Requirements checklist. Do **not** proceed past a missing account, a

missing connector, missing least-privilege roles, or a missing SHIR/network path — remediate those first.

Step 2 — Design the collection and role model

1. Decide the **collection** the Google Cloud sources register into. For the pilot, create one

collection named `DGGCP-Collection-Pilot` under the root; plan the broader hierarchy (for example a `GCP` branch by project or by sensitivity) on paper for the expansion phase.

1. Assign least-privilege roles at the narrowest scope: **Data source admin** on

`DGGCP-Collection-Pilot` to whoever will register/scan Google Cloud sources, **Data curator** to the catalog editors, **Data reader** to browsers, and **Collection admin** to a small named group. Reserve account-level admin for a minimal named group.

Step 3 — Establish the BigQuery service-account credential (safe by design)

1. In **Google Cloud**, create a **service account** in the in-scope project and grant it **read-only**

BigQuery access — historically **BigQuery Data Viewer** + **BigQuery Metadata Viewer** plus read-job ability (**BigQuery Job User**); **verify the current required roles** for the Purview connector. Enable the **BigQuery API** in the project.

1. Generate a **service-account key** for it. **Do not place the key in a script or document.**

2. In Azure, link an **Azure Key Vault** to the Purview account (or confirm one is linked) and store

the service account's **private key** as a Key Vault secret.

1. In Purview, register a credential **by name** (for example `DGGCP-Cred-BQ-KV`) of the BigQuery

service-account type that references the Key Vault secret. This name is what the scripts reference; no key material is ever inlined in a script.

Step 4 — Establish the SHIR and network path

1. Install and register a **self-hosted integration runtime (SHIR)** to the Purview account on a

machine with **network line of sight to Google Cloud / BigQuery**.

1. Install the **BigQuery driver prerequisites** on the SHIR host (historically a **Java Runtime

Environment / JDK **for the JDBC driver —** verify the current version** for your tenant).

1. Confirm the SHIR shows **Running/Available** and that any Google Cloud / network firewall rules

permit the SHIR to reach BigQuery.

Step 5 — Record the pilot scope

1. Run `DGGCP-Set-Scope.ps1` to record the pilot Google Cloud source set (one BigQuery project / one

or two datasets) with owners, plus the pilot collection and scan-rule-set intent (read-only; produces the scope manifest you register against).

Step 6 — Register the pilot Google Cloud source into the Data Map

1. Register the pilot source into `DGGCP-Collection-Pilot`. You may use the portal

(**Register** → **Google BigQuery**), or run `DGGCP-Register-GcpSource.ps1` which **defaults to** `-WhatIf` and prints exactly what it would register (kind `GoogleBigQuery`, the project ID, the collection, and the credential name); re-run with `-Commit` to register.

1. Run **Test connection** for the registered source and confirm it passes (service-account auth over the SHIR succeeds) before scanning. Do not scan until Test connection passes.

Step 7 — Configure and pilot the scan (bounded scope, pilot-first)

1. Define the scan configuration: a **scan rule set** named `DGGCP-ScanRuleSet-Pilot` (start from the system default rule set), the **classifications** to apply (system classifications plus any custom rule named `DGGCP-Classification-Custom-<name>`), and a **scoped** scan that targets only the pilot datasets — not every dataset in the project.

1. Trigger the scan **once** on the bounded scope (not scheduled yet) via `DGGCP-Register-GcpSource.ps1 -ConfigureScan` (`-WhatIf` by default; `-Commit` to apply) or the portal. Pass/fail: the scan completes with **status Succeeded** and returns assets with classifications.

1. **Rollback:** if a scan misbehaves, over-reaches, or runs up BigQuery query/egress cost, cancel the run in the portal and set the scan to disabled; a scan does not modify source data, but cancel to stop resource/cost consumption. Re-scope and re-run on a smaller set.

Step 8 — Review scanned assets and classifications

1. Run `DGGCP-Report.ps1` to list the pilot Google Cloud assets discovered (BigQuery datasets/tables/columns), the classifications applied, and any assets returned **unclassified**.

1. With the data owner, confirm the classifications are correct. Where a real internal pattern was missed, define a **custom classification rule** (`DGGCP-Classification-Custom-<name>`), validate it against sample data, add it to `DGGCP-ScanRuleSet-Pilot`, and re-scan the pilot scope.

Step 9 — Assign ownership on the Google Cloud assets

1. In the catalog, assign the **data owner** and **steward** recorded in the scope manifest to the scanned datasets/tables so every governed Google Cloud asset has a named owner — ownership is what makes classification and later access decisions accountable.

Step 10 — Validate against the pilot gate

1. Complete the Testing Guide. Run `DGGCP-Test-Readiness.ps1` for a consolidated PASS/GAP gate check.

The pilot passes only when: the Google Cloud source is registered and **Test connection** passes; the scan **Succeeded** and returned classified assets; and every scanned pilot asset has an owner assigned.

1. Remediate any gaps and re-run the relevant validation before proceeding. Do **not** widen scope until the gate passes.

Step 11 — Expand across the Google Cloud estate (enforce)

1. After the pilot passes, register further BigQuery projects/datasets one grouping at a time (for

example per project), re-running `DGGCP-Set-Scope.ps1`, `DGGCP-Register-GcpSource.ps1`, and `DGGCP-Report.ps1` for each new scope. Reuse the same service-account pattern (extending its read-only grant to the new datasets/projects) and Key Vault credential pattern.

1. Move scans from one-time to **scheduled** only once a scope is validated, so classifications stay current as Google Cloud data changes — and watch BigQuery query/egress cost as scope grows.
1. Keep ownership assigned on every newly scanned asset and track coverage with `DGGCP-Report.ps1`.

Order of operations

```
Assess -> Design (collection/roles) -> BigQuery service-account credential (Key Vault) -> SHIR & network -> Record scope -> Register GCP source -> Pilot scan (bounded) -> Review classifications -> Assign ownership -> Validate gate -> Expand (scheduled scans)
```

5. Testing Guide — Google Cloud Data-Source Governance: Microsoft Purview

Run these tests after the pilot scan (Implementation Steps 6–9) and again after each expansion scope. Each test confirms one part of the Google Cloud governance workflow works end to end. All read-only checks are safe to repeat.

Test 1 — Account and roles are ready

Do: Run `DGGCP-Discovery.ps1`. Expected: The Purview account endpoint is reachable, `DGGCP-Collection-Pilot` exists, and least-privilege roles (Collection admin, Data source admin, Data curator, Data reader) are reported as assigned; **Data source admin** (needed to register/scan) is present. No required role is missing.

Test 2 — The Google BigQuery connector is available

Do: In the Purview portal, open **Data Map** → **Sources** → **Register**. Expected: **Google BigQuery** appears as a registerable source type for your tenant.

Test 3 — The service-account credential path works (Key Vault + Test connection)

Do: On the registered BigQuery source, run **Test connection**; confirm the credential references the Key Vault secret **by name** and the BigQuery API is enabled in the project. Expected: Purview authenticates to BigQuery as the **service account** using the **named Key Vault credential** and **Test connection passes**. No key material is stored anywhere.

Test 4 — The SHIR and network path work

Do: Confirm the **self-hosted integration runtime (SHIR)** shows **Running/Available** and that the Java/driver prerequisites are installed on the SHIR host. Expected: The SHIR is Running with line of sight to BigQuery; **Test connection** on the source succeeds through it.

Test 5 — The scan succeeds and returns assets

Do: Review the pilot scan run in the portal and run `DGGCP-Report.ps1`. Expected: The scan status is **Succeeded**, the run scanned the intended bounded scope (the pilot datasets, not the whole project), and assets (BigQuery datasets/tables) were returned into the Data Map.

Test 6 — Google Cloud assets are classified

Do: Run `DGGCP-Report.ps1` and review the classification coverage for the pilot Google Cloud assets. Expected: System (and any custom) classifications are applied to the assets that contain matching data; the count of unexpectedly **unclassified** assets is understood and accepted by the data owner.

Test 7 — Scanned Google Cloud assets have owners

Do: Open the pilot datasets/tables in the catalog and run `DGGCP-Report.ps1`. Expected: Every scanned pilot asset has a named **owner/steward** assigned (from the scope manifest); no governed Google Cloud asset is unowned.

Test 8 — Consolidated readiness gate

Do: Run `DGGCP-Test-Readiness.ps1 -PurviewAccount <account> -Collection DGGCP-Collection-Pilot`. Expected: A single PASS/GAP summary confirming account reachable, roles present, the BigQuery source registered and Test-connection-ready, at least one scan Succeeded, classification coverage > 0, and owners assigned — the pilot gate in one view.

Pass/fail gate

Gate	Requirement
Pilot pass	Tests 1–7 pass: connector available, service-account Key Vault credential Test connection passes over a Running SHIR, scan Succeeded with classified assets, and every scanned asset has an owner (Test 8 shows all PASS)
Expand pass	Tests 1–7 pass for each new Google Cloud scope, and scheduled scans are enabled only after the scope validates (with expected BigQuery query/egress cost accepted)

6. Troubleshooting Guide — Google Cloud Data-Source Governance: Microsoft Purview

Use `DGGCP-Discovery.ps1` for readiness/role/runtime state, `DGGCP-Report.ps1` for scan and classification coverage, `DGGCP-Set-Scope.ps1` to (re)record a scope, `DGGCP-Register-GcpSource.ps1` (`-WhatIf` by default) to register/scan, and `DGGCP-Test-Readiness.ps1` for the consolidated gate. All scripts are read-only unless you pass `-Commit` (only `DGGCP-Register-GcpSource.ps1` writes).

A. Cannot reach the account / token fails

Symptom	Cause	Remediation
`Discovery` reports endpoint GAP	Wrong account name or endpoint	Confirm the account name; the endpoint is `https://<account>.purview.azure.com`; re-run `DGGCP-Discovery.ps1`
Token acquisition fails	Not signed in / wrong resource	Run `Connect-AzAccount`; the scripts request a token for `https://purview.azure.net` — confirm your account can obtain it
401/403 on every call	Missing least-privilege role	Assign Data reader (browse) or the specific role the action needs at the collection scope; re-run
Calls fail with "api-version" errors	The example API version has moved	The Data Governance experience is evolving — verify and set the current data-plane API version for your tenant

B. Google BigQuery source registration or Test connection fails

Symptom	Cause	Remediation
Register returns 403	Not a Data source admin on the collection	Assign Data source admin on `DGGCP-Collection-Pilot`; re-run `DGGCP-Register-GcpSource.ps1 -Commit`
Unknown source "kind"	Connector name/availability differs	Verify the exact BigQuery connector `kind` and its availability (the connector may be preview) in Microsoft's docs; **Data Map** → **Sources** → **Register** must offer Google BigQuery
"Credential not found"	Key Vault credential name wrong or vault not linked	Link the Key Vault to the account and reference the credential **by name** (`-KeyVaultCredential DGGCP-Cred-BQ-KV`); never inline a key

Symptom	Cause	Remediation
Test connection fails — auth	Service account lacks read / wrong key / key expired	Grant the service account read-only BigQuery roles (Data Viewer + Metadata Viewer + Job User — verify), regenerate/re-store the private key in Key Vault, confirm the BigQuery API is enabled; re-test
Test connection fails — no runtime	No self-hosted integration runtime (SHIR)	BigQuery scans require a **Running SHIR**; install/register one with line of sight to BigQuery and reference it with `-IntegrationRuntime`; re-test
Test connection fails — driver	Java/JDBC driver prerequisites missing on SHIR	Install the required Java/JDK and BigQuery driver prerequisites on the SHIR host (verify the current version); restart the SHIR; re-test

C. Scan runs but classifies nothing

Symptom	Cause	Remediation
Assets returned, 0 classified	Scan rule set has no classifications	Confirm `DGGCP-ScanRuleSet-Pilot` includes system classifications; re-scan and review with `DGGCP-Report.ps1`
An internal pattern isn't detected	No custom classification rule	Define `DGGCP-Classification-Custom-<name>`, validate against sample data, add to the rule set, re-scan
Scan scope too broad / slow / costly	Whole project scanned instead of pilot datasets	Re-scope the scan to the bounded pilot datasets; keep it one-time until validated; watch BigQuery query/egress cost
Scan status Failed	Transient / auth / SHIR / network	Open the run in the portal for the failure reason; confirm the SHIR is Running and the service-account credential resolves; re-run

D. Classifications look wrong

- False positives/negatives are expected on first scan. Review with the **data owner** using

`DGGCP-Report.ps1`; tune or remove a custom classification rule and re-scan the pilot scope. The solution provides the technical result — the owner decides whether a classification is correct for the business.

E. Ownership / catalog issues

Symptom	Cause	Remediation
Scanned dataset has no owner	Owner not assigned	Assign the owner/steward from the scope manifest in the catalog; every governed asset must have a named owner
Cannot edit catalog metadata	Not a Data curator	Assign Data curator at the collection scope; re-try

F. SHIR health

• BigQuery scans depend on the self-hosted integration runtime. If scans queue or fail intermittently,

confirm the SHIR shows **Running/Available**, the host has network line of sight to BigQuery, and the Java/driver prerequisites are current. `DGGCP-Discovery.ps1` reports whether a self-hosted runtime is registered.

G. Cost surprises

• Scanning issues read jobs against BigQuery and can incur **query/egress cost**. If cost climbs,

cancel the run in the portal, re-scope to the bounded pilot datasets, keep scans one-time until a scope validates, and enable a schedule only for validated scopes. Agree expected cost with the source owner before widening.

H. Diagnostics quick reference

```
.\DGGCP-Discovery.ps1 -PurviewAccount contoso-purview                     # readiness: endpoint, roles, S
.\DGGCP-Set-Scope.ps1 -ProjectId my-gcp-project -Source @('my-gcp-project/analytics_ds')  # record a pilot
.\DGGCP-Register-GcpSource.ps1 -PurviewAccount contoso-purview -Collection DGGCP-Collection-Pilot `
    -SourceName bq-analytics -ProjectId my-gcp-project                    # -WhatIf by default; add -Comm
.\DGGCP-Report.ps1 -PurviewAccount contoso-purview -Collection DGGCP-Collection-Pilot     # scan + classif
.\DGGCP-Test-Readiness.ps1 -PurviewAccount contoso-purview -Collection DGGCP-Collection-Pilot  # consolida
```

I. When to escalate

If the account is reachable, roles are assigned, the SHIR is Running with the driver prerequisites, Test connection passes, and a scoped scan still returns no classified assets after tuning the rule set, capture the `DGGCP-Discovery.ps1` and `DGGCP-Report.ps1` output and the portal scan-run detail, and review the BigQuery connectivity and rule set with the data owner and your Purview/Azure administrator. For evolving or preview features, verify current availability, required GCP IAM roles, and API versions against Microsoft's documentation before assuming a defect.

7. Best Practices — Google Cloud Data-Source Governance: Microsoft Purview

• Design the collection hierarchy before you register anything. Collections carry both organization

and access control; a hierarchy planned by GCP project or sensitivity (with `DGGCP-Collection-Pilot` as the first node) is far cheaper than re-parenting sources later.

• Least-privilege by collection, always. Assign Data source admin, Data curator, and Data reader at the narrowest collection scope; reserve account-level Collection admin for a small named group. Broad roles at the root are the most common governance finding.

• Never put key material in a script or scan. Grant the Google Cloud **service account read-only** BigQuery access, store its **private key in an Azure Key Vault** linked to the account, and reference the credential **by name** — this is both the secure pattern and what keeps the provided scripts secret-free. Rotate the service-account key on a schedule and re-store it in Key Vault.

• Pilot every scan on a bounded scope. Scoped, one-time scans on a pilot dataset set prove classification and BigQuery cost before you point a scan at an entire project or the whole estate; move to scheduled scans only after a scope validates.

• Keep the SHIR healthy and reachable. BigQuery scans run on the self-hosted integration runtime; keep its Java/driver prerequisites current, monitor that it stays **Running**, and confirm its network line of sight to BigQuery before each expansion wave.

• Trust system classifications first, add custom sparingly. The built-in classifications cover most regulated data; define a custom classification rule (`DGGCP-Classification-Custom-<name>`) only for a real internal pattern, and validate it against sample data before estate-wide use.

• Every scanned asset has an owner and a steward. Ownership is what makes classification and access decisions accountable; an unowned BigQuery dataset in the catalog is a governance gap, not a governed asset.

• Watch BigQuery cost as scope grows. Scanning issues read jobs against BigQuery and can incur query/egress cost; scope scans to what needs classifying, schedule only validated scopes, and agree the expected cost with the source owners before widening.

• Treat determinations as someone else's job. The solution classifies, maps, and catalogs; the business/records/legal owners decide sensitivity, retention, and access. Keep that separation explicit so technical scanning is never mistaken for a compliance ruling.

• Verify licensing and portal paths as you go. The Data Governance experience, the Google BigQuery connector, the required GCP IAM roles, and its consumption billing are evolving — confirm current availability and pricing in Microsoft's docs and the portal rather than assuming a path, a role, or a price is permanent.

8. Deliverables Summary — Google Cloud Data-Source Governance: Microsoft Purview

Documentation

Document	Purpose
Executive Summary	Business case and what's in the box
1. Solution Overview	The Data Map, the Google BigQuery connector, scanning + classification, collections & roles, and how it complements M365 Purview (the why)
2. Requirements Guide	Every prerequisite with Verify / Implement-if-missing
Global Requirements (Purview)	Category-wide prerequisites, published once
3. Implementation Guide	Exact, ordered steps: design, service-account credential, SHIR/network, register, pilot-scan, review, assign owners, validate, expand
5. Testing Guide	Validation procedures with pass/fail gates
6. Troubleshooting Guide	Field issues and remediation
7. Best Practices	Field-proven recommendations

Named artifacts

Artifact	Name
Pilot collection	`DGGCP-Collection-Pilot`
Scan rule set	`DGGCP-ScanRuleSet-Pilot`
BigQuery service-account credential (Key Vault, by name)	`DGGCP-Cred-BQ-KV`
Custom classification rule (as needed)	`DGGCP-Classification-Custom-<name>`

PowerShell scripts

Script	What it does
`DGGCP-Discovery.ps1`	Reports Google Cloud governance readiness: account endpoint reachable, collections + role coverage, registered integration runtimes (SHIR for BigQuery), and registered data-source count (Purview data-plane REST via an Azure token; read-only)
`DGGCP-Set-Scope.ps1`	Records the pilot/expansion scope — the BigQuery project/dataset set with owners and the scan-rule-set intent — as a local scope manifest to register against (read-only)
`DGGCP-Register-GcpSource.ps1`	Registers a Google BigQuery source and configures a scoped scan; **defaults to** `-WhatIf` (prints the plan), applies only with `-Commit`; references the Key Vault credential by name — no secrets in code

Script	What it does
`DGGCP-Report.ps1`	Scan + classification coverage report: per-source last scan status, classified vs unclassified assets with coverage %, and owner coverage (read-only)
`DGGCP-Test-Readiness.ps1`	Consolidated PASS/GAP pilot-gate check: account reachable, roles present, source registered, scan Succeeded, classification coverage > 0, owners assigned (read-only)

What the buyer receives

- This implementation guide as a branded Word document and PDF.
- All five PowerShell scripts in a single ZIP.
- A repeatable method following the model: Assess → Design → Configure → Pilot → Validate → Expand.

Outcome

Google Cloud data — BigQuery projects, datasets, and tables — registered, scanned, and classified in the same Purview Data Map that governs the rest of the estate, with named owners and a pilot-proven scan pattern before you scale.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

DGGCP-Discovery.ps1

Purpose: Reports readiness before registering the Google BigQuery source or scanning: whether the Microsoft Purview account data-plane endpoint is reachable, which collections exist and their role-assignment coverage, which integration runtimes are registered (BigQuery scans require a self-hosted integration runtime / SHIR), and a high-level count of registered data sources. Read-only — it makes no change to the account, sources, or catalog. Supports the Assess step. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGGCP-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (need a SHIR for BigQuery), and registered data-source count — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Data Governance (Google BigQuery). Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, the Google BigQuery connector, and the endpoints/API versions are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGGCP-Discovery.ps1 -PurviewAccount contoso-purview Requires the Az PowerShell modules (Az.Accounts). Sign in first with Connect-AzAccount using an account holding at least Data reader on the collections. The script acquires a data-plane token for the Purview REST API (resource https://purview.azure.net); no secret is stored in the script. EXPECTED RESULTS A readiness summary: endpoint reachable (PASS/GAP), collection count and per-collection role coverage, registered integration runtimes (need a SHIR for BigQuery), and registered data-source count — with a PASS/GAP line per signal to record against the Requirements checklist. .NOTES Purview — Data Governance (Google BigQuery). Read-only against the Purview data-plane REST API. Verify your licensing/tenant: the Data Governance experience, the Google BigQuery connector, and the endpoints/API versions are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)] [string]$PurviewAccount, # short account name, e.g. contoso-purview [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGGCP-Register-GcpSource.ps1

Purpose: Registers a Google BigQuery data source into a Data Map collection and (with -ConfigureScan) prepares a scoped, one-time scan using a named scan rule set. SAFE BY DEFAULT: with no -Commit it runs in -WhatIf mode and only prints the exact registration/scan payload it WOULD send — it changes nothing. Pass -Commit to apply. No credential is ever placed in this script; BigQuery authentication uses a Google Cloud service-account key stored in Azure Key Vault and referenced by name, and the scan runs on a self-hosted integration runtime (SHIR). HOW TO RUN Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\DGGCP-Register-GcpSource.ps1 -PurviewAccount contoso-purview `-Collection DGGCP-Collection-Pilot -SourceName bq-analytics -ProjectId my-gcp-project Apply (register + configure scan): ... same args ... -ConfigureScan -ScanRuleSet DGGCP-ScanRuleSet-Pilot` -KeyVaultCredential

DGGCP-Cred-BQ-KV -IntegrationRuntime my-shir -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with DGGCP-Report.ps1. .NOTES Purview — Data Governance (Google BigQuery). Writes gated behind -Commit; -WhatIf is the default. Verify the exact BigQuery source "kind", the required credential type, and current scan API versions for your connector in Microsoft's docs — the multicloud connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$ProjectId, # GCP project ID holding the BigQuery datasets [string]$Kind = 'GoogleBigQuery', # verify the exact connector kind for your tenant [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGGCP-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$IntegrationRuntime, # NAME of the self-hosted IR (SHIR) that runs the scan [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

How to run: Preview (default, no change): powershell -ExecutionPolicy Bypass -File .\DGGCP-Register-GcpSource.ps1 -PurviewAccount contoso-purview `-Collection DGGCP-Collection-Pilot -SourceName bq-analytics -ProjectId my-gcp-project` Apply (register + configure scan): ... same args ... `-ConfigureScan -ScanRuleSet DGGCP-ScanRuleSet-Pilot` -KeyVaultCredential DGGCP-Cred-BQ-KV -IntegrationRuntime my-shir -Commit Requires Az.Accounts and Connect-AzAccount with Data source admin on the collection. EXPECTED RESULTS Default: a printed "[WhatIf] would register / would configure scan" plan and NO change. With -Commit: the source is registered (HTTP 200/201) and, with -ConfigureScan, a scan definition is created; the script prints the resulting object names. Trigger/monitor the scan run in the portal or with DGGCP-Report.ps1. .NOTES Purview — Data Governance (Google BigQuery). Writes gated behind -Commit; -WhatIf is the default. Verify the exact BigQuery source "kind", the required credential type, and current scan API versions for your connector in Microsoft's docs — the multicloud connectors are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [Parameter(Mandatory = $true)][string]$Collection, [Parameter(Mandatory = $true)][string]$SourceName, [Parameter(Mandatory = $true)][string]$ProjectId, # GCP project ID holding the BigQuery datasets [string]$Kind = 'GoogleBigQuery', # verify the exact connector kind for your tenant [switch]$ConfigureScan, [string]$ScanRuleSet = 'DGGCP-ScanRuleSet-Pilot', [string]$KeyVaultCredential, # NAME of a Key Vault credential already linked to the account [string]$IntegrationRuntime, # NAME of the self-hosted IR (SHIR) that runs the scan [string]$Endpoint = "https://$PurviewAccount.purview.azure.com", [switch]$Commit)

DGGCP-Report.ps1

Purpose: Reports the health of the governed Google Cloud estate for a scope: latest scan run status per BigQuery source, counts of classified vs unclassified assets, and owner coverage. Read-only — it queries the Purview data-plane REST / catalog search and changes nothing. Use it to review the pilot (Testing Guide) and to track coverage as the estate expands. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGGCP-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGGCP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and owner coverage — PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Data Governance (Google BigQuery).

Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGGCP-Report.ps1 -PurviewAccount contoso-purview ` -Collection DGGCP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A coverage summary: per-source last scan status (Succeeded/Failed/Running), classified vs unclassified asset counts with a classification-coverage %, and owner coverage — PASS/GAP flags you can record against the pass/fail gate. .NOTES Purview — Data Governance (Google BigQuery). Read-only. Verify current search/scan API versions for your tenant; the Data Governance experience and its endpoints are evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection, [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

DGGCP-Set-Scope.ps1

Purpose: Captures and writes a scope manifest for the pilot (or an expansion wave): the target collection, the BigQuery project/dataset source set to register (each with an owner and steward), the scan rule set to apply, and the credential name to reference. Read-only against the tenant — it writes only a local JSON manifest you register and scan against; it does not change the Purview account and stores no credential. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGGCP-Set-Scope.ps1 -Collection DGGCP-Collection-Pilot `-ProjectId my-gcp-project -Source @('my-gcp-project/analytics_ds','my-gcp-project/finance_ds')` -Credential DGGCP-Cred-BQ-KV -OutFile .\pilot-scope.json EXPECTED RESULTS A pilot-scope.json manifest listing the collection, GCP project, BigQuery dataset sources, scan rule set, and credential name, plus a printed summary — used as the input to DGGCP-Register-GcpSource.ps1. .NOTES Purview — Data Governance (Google BigQuery). Local manifest only; no tenant change; no secrets. Names are white-label placeholders (contoso / example project IDs). > [CmdletBinding()] param([string]$Collection = 'DGGCP-Collection-Pilot', [Parameter(Mandatory = $true)][string]$ProjectId, # GCP project ID holding the BigQuery datasets [string[]]$Source = @('my-gcp-project/analytics_ds'),# project/dataset entries in scope [string]$ScanRuleSet = 'DGGCP-ScanRuleSet-Pilot', [string]$Credential = 'DGGCP-Cred-BQ-KV', # NAME of the Key Vault credential (never a secret) [string]$OutFile = '.\pilot-scope.json')

How to run: powershell -ExecutionPolicy Bypass -File .\DGGCP-Set-Scope.ps1 -Collection DGGCP-Collection-Pilot `-ProjectId my-gcp-project -Source @('my-gcp-project/analytics_ds','my-gcp-project/finance_ds')` -Credential DGGCP-Cred-BQ-KV -OutFile .\pilot-scope.json EXPECTED RESULTS A pilot-scope.json manifest listing the collection, GCP project, BigQuery dataset sources, scan rule set, and credential name, plus a printed summary — used as the input to DGGCP-Register-GcpSource.ps1. .NOTES Purview — Data Governance (Google BigQuery). Local manifest only; no tenant change; no secrets. Names are white-label placeholders (contoso / example project IDs). > [CmdletBinding()] param([string]$Collection = 'DGGCP-Collection-Pilot', [Parameter(Mandatory = $true)][string]$ProjectId, # GCP project ID holding the BigQuery datasets [string[]]$Source = @('my-gcp-project/analytics_ds'),# project/dataset entries in scope [string]$ScanRuleSet = 'DGGCP-ScanRuleSet-Pilot', [string]$Credential = 'DGGCP-Cred-BQ-KV', # NAME of the Key Vault credential (never a secret) [string]$OutFile = '.\pilot-scope.json')

DGGCP-Test-Readiness.ps1

Purpose: Runs the pilot gate in one view: account endpoint reachable, least-privilege roles present, a self-hosted integration runtime (SHIR) registered, the Google BigQuery source registered, at least one scan Succeeded, classification coverage > 0, and owners assigned. Read-only — it only reads the Purview data-plane REST / catalog search and prints a single PASS/GAP summary. It does not register, scan, or change anything. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\DGGCP-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGGCP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A PASS/GAP line per gate signal and an overall PILOT GATE: PASS only when every signal passes. Use it as the Step 10 gate before widening scope. .NOTES Purview — Data Governance (Google BigQuery). Read-only. Verify current API versions and the BigQuery connector availability for your tenant; the Data Governance experience is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGGCP-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")

How to run: powershell -ExecutionPolicy Bypass -File .\DGGCP-Test-Readiness.ps1 -PurviewAccount contoso-purview ` -Collection DGGCP-Collection-Pilot Requires Az.Accounts + Connect-AzAccount with at least Data reader on the collection. EXPECTED RESULTS A PASS/GAP line per gate signal and an overall PILOT GATE: PASS only when every signal passes. Use it as the Step 10 gate before widening scope. .NOTES Purview — Data Governance (Google BigQuery). Read-only. Verify current API versions and the BigQuery connector availability for your tenant; the Data Governance experience is evolving. > [CmdletBinding()] param([Parameter(Mandatory = $true)][string]$PurviewAccount, [string]$Collection = 'DGGCP-Collection-Pilot', [string]$Endpoint = "https://$PurviewAccount.purview.azure.com")